

Homework 3: RAG for Science Question Answering

314831014 呂茗憲 [Github 網址](#)

1. 分塊大小分析 (Chunk Size Analysis)：對上下文完整性的影響

Chunk size 顯著影響了系統捕捉完整答案並提供給 LLM 的能力。

Part 1 – Chunking Strategy Comparison		
Metric	Method A	Method B
Chunk Size	500 tokens	1000 tokens
Overlap	50 tokens (10%)	100 tokens (10%)
Splitting Strategy	Fixed-size	Semantic / Recursive
Total Chunks	380	212
Avg Chunks / Document	2.5	1.4
Knowledge Corpus	150 documents	150 documents

- Method A (500 tokens)：**較小的 chunk 容易在句子中間截斷，並破壞選擇題的結構完整性。在此策略下，150 份文件被切分為 380 個 chunk（平均每份文件被切為 2.5 塊）。這導致一種常見的截斷現象：一個 chunk 可能只包含題目與前三個選項，而剩餘的選項被孤立在下一個 chunk 中。這剝奪了 LLM 評估所有潛在答案所需的完整上下文。
- Method B (1000 tokens)：**對於此特定資料集，較大的 chunk 表現優異許多。它僅產生 212 個 chunk（平均每份文件 1.4 塊）。這種切法通常能將完整的題目與所有對應選項保留在同一個 chunk 中。這種語意上的完整性直接避免了句子被截斷的問題，並為 LLM 提供了結構完整的推理依據。

2. 重排序影響 (Re-ranking Impact)：修正檢索失敗案例

比較 "Vector Search Only" 與 "Vector Search + Re-ranking" 的 Hit Rate:

Part 2 – Retrieval Hit Rate Comparison (N=20 eval questions, top-k=3)				
Approach	Hit Rate	Hits	Avg Vec (s)	Avg Rerank (s)
Vector Search Only	90.00%	18/20	0.020s	N/A
Vector Search + Re-ranking	90.00%	18/20	0.012s	0.010s
Delta	+0.00%	–	–	–

引入第二階段的 Cross-Encoder 成功修正了第一階段 Dense Retrieval 未能將最佳上下文排在首位的兩個案例。

```
Re-ranking Example 1 – Peierls bracket
- Query: What is the Peierls bracket in canonical quantization?
- Correct Answer: C. The Peierls bracket is a Poisson bracket derived from the action in the canonical quantization method that converts the quotient algebra into a Poisson algebra.
- BEFORE (Vector Top-1, relevance 75%): 只有單一錯誤選項片段 - "E. The Peierls bracket is a mathematical tool used to generate the Euler-Lagrange equations..."
- AFTER (Re-rank Top-1, score=10.08, relevance 100%): 完整問題 block 含所有選項 A-D，正確答案 C 完整出現在 context 中
```

• 範例 1：Peierls bracket

- **Dense Retrieval (Vector Search) (Top-1)**：檢索到一個破碎的 chunk，僅包含一個錯誤選項片段 ("E. The Peierls bracket is a mathematical tool used...")，相關度僅 75%，遺漏了核心定義。
- **Cross-Encoder (Re-ranking) (Top-1)**：Cross-Encoder 成功辨識出包含所有選項 (A~D) 的完整問題區塊，並給予 +10.08 的高分。這成功將包含正確答案的 chunk ("C. The Peierls bracket is a Poisson bracket derived from...") 提升至首位，提供 LLM 100% 相關的上下文。

```
Re-ranking Example 2 – Reciprocal length
- Query: What is reciprocal length or inverse length?
- Correct Answer: E. Reciprocal length or inverse length is a quantity or measurement used in several branches of science and mathematics...
- BEFORE (Vector Top-1, relevance 0%): "D. An improper rotation is the combination of a rotation about an axis..." - 與查詢完全無關
- AFTER (Re-rank Top-1, score=-7.64, relevance 25%): 含 "length contraction" 等相關物理量詞的 chunk，Cross-Encoder 把 0% 相關的錯誤文件替換掉
```

• 範例 2：Reciprocal length

- **Dense Retrieval (Vector Search) (Top-1)**：檢索到一篇探討「不當旋轉 (improper rotation)」的完全無關文件 (相關度 0%)。

- **Cross-Encoder (Re-ranking) (Top-1)** : Cross-Encoder 識別出此語意落差並將該雜訊過濾。它將一個探討相關物理測量 ("length contraction") 的 chunk 提升至首位 (分數 : -7.64)。雖然不是完美的精確匹配 (相關度 25%)，但這展示了模型過濾 0% 相關雜訊並優化基準上下文的能力。

3. 延遲分析 (Latency Analysis) : 重排序的時間成本值得嗎 ?

為了評估系統額外負擔，針對 50 個評估問題測量了平均每次查詢的延遲：

Part 3 - Generation Accuracy & Latency (50 eval questions, Mistral-7B-Instruct-v0.3)		
Metric	Vector Only	With Re-ranking
Correct	27 / 50	27 / 50
Accuracy	54.00%	54.00%
Vector Search	0.013s	0.019s
Re-ranking	—	0.013s
LLM Generation	0.883s	1.203s
Total / Query	0.896s	1.235s

- **Dense Retrieval (Vector Search) :**

0.013s (Vector Search Only) / 0.019s (Vector Search + Re-ranking)

- **Cross-Encoder (Re-ranking) :**

None (Vector Search Only) / 0.013s (Vector Search + Re-ranking)

- **LLM Generation (Mistral-7B) :**

0.883s (Vector Search Only) / 1.203s (Vector Search + Re-ranking)

結論： 在這次作業的特定實作環境下，Re-ranking 所增加的時間成本並不值得。引入 Re-ranking 將每次查詢的總處理時間從 0.896s 增加至 1.235s (增加了 0.339s，即 +38% 的延遲)。這增加的延遲有很大一部分來自 LLM 生成階段；由於 Re-ranking 傾向將較長、語意更完整的 chunk (來自 Method B) 推升至首位，導致 LLM 必須處理更大的上下文窗口。

儘管付出了 38% 的延遲代價，Accuracy 依然停滯在 54.00%，且基礎的 Hit Rate 在未加 Re-ranking 前就已達到最佳的 90.00%。對於僅有 150 份文件的微型語料庫而言，Vector Search 本身已具備極高的效率。在此情境下，Re-ranking 的邊際效益趨近於零；不過，若將其部署於資料量龐大、雜訊極高且語意重疊度大的語料庫（如完整的維基百科）中，Re-ranking 提升精準度的價值才有可能超過延遲代價。

4. 心得與反思 (Conclusion & Reflection)

本次實作深刻印證了 RAG 是一個高度依賴權衡的工程實踐，而非單純的 API 串接。從確保上下文完整的語意分塊（Chunking）、面臨領域適配挑戰的重排序模型（Re-ranking），到約束模型幻覺的提示詞設計，每個環節都緊密相扣。實驗結果也提醒我應避免在微型資料庫中盲目堆疊複雜的排序架構，只會徒增延遲而無法提升準確率。我覺得未來唯有緊扣資料規模與領域特性來客製化檢索策略，才能打造出真正高效且精準的 RAG 系統。